

ATHREIX • NEXUS

Athreix Nexus

Never explain your project to an AI twice.

A local-first, offline-first, cross-platform command-line tool that acts as a persistent memory layer for software projects. Where Git stores source code, Nexus stores project knowledge — decisions, architecture, timeline, and the context an AI assistant needs to continue your work.

DOCUMENT

Technical Report & Command Reference

DESCRIPTION

TLDR

AUTHOR

Tushar Dhokane / Athreix

DATE

8 June 2026

LICENSE

MIT

NOTES

1st 2021 edition — 13-page workshop

— Contents

1	Executive Summary	
2	Philosophy & Positioning	
3	System Architecture	
4	Project Layout	
5	Command Reference (complete)	
6	Data & Persistence	
7	NXP — Nexus Protocol	
8	Reliability & Recent Enhancements	
9	Build, Test & Install	
10	Roadmap	

1 Executive Summary

What Nexus is, and the problem it solves.

Modern software development increasingly happens alongside AI assistants. Yet every new chat session starts from zero: the developer re-explains the architecture, the decisions already made, the current task, and the constraints. **Athreix Nexus eliminates that repetition.** It maintains a durable, structured memory of a project on disk and reconstructs a complete continuation context on demand.

Local & Offline
Zero network calls. All memory lives in a `.nexus/` folder beside your code.

Universal Handoff
Generates context compatible with Claude, ChatGPT, Gemini, Cursor, Windsurf and open agents.

Structured Memory
Decisions, architecture, timeline, tasks, and a knowledge graph — not free-form notes.

Fast & Portable
Single optimized Rust binary (~10 MB), SQLite + JSON storage, cross-platform.

Git stores	Nexus stores
Source code	Project knowledge
Commits & diffs	Decisions & rationale
Branches	Timeline, tasks & AI sessions

2 Philosophy & Positioning

Nexus is deliberately **not** a SaaS dashboard, a project-management tool, or a note app. It is infrastructure.

Git for AI Context

Version-controlled, portable project understanding.

Memory OS

A persistent knowledge layer for software projects.

Handoff Infrastructure

A universal protocol for transferring context to any AI.

Terminal-native

Everything works from the CLI. Offline. Local. Cross-platform.

3 System Architecture

A 13-crate clean architecture in Rust, with strict layer separation.

Crate	Responsibility
nexus-cli	CLI layer — argument parsing (clap) and command dispatch
nexus-core	Shared types, models, paths, and error definitions
nexus-memory	Memory engine — SQLite + JSON persistence, atomic writes
nexus-context	Context engine — continuation-context reconstruction
nexus-knowledge	Knowledge engine — graph of files, decisions, tasks, packages
nexus-git	Git engine — branch, commit history, dirty-file summary (git2)
nexus-scanner	Scanner engine — tree-sitter parsing & dependency detection
nexus-decision	Decision engine — architectural decision records (ADR)
nexus-snapshot	Snapshot engine — memory checkpoints & restore
nexus-handoff	Handoff engine — universal AI continuation prompt
nexus-compression	Compression engine — token-budget context reduction
nexus-search	Search engine — SQLite FTS5 full-text memory search
nexus-nxp	NXP engine — portable Nexus Protocol export/import

Technology Stack

Rust 2021

clap

rusqlite + FTSS

tree-sitter

git2

serde / serde.json

rayon

chrono

uuid

sha2

Tree-sitter grammars: Rust, JavaScript, TypeScript, Python. Release profile uses LTO, a single codegen unit, and symbol stripping for a compact binary.

4 Project Layout

Every Nexus-enabled project contains a single `.nexus/` directory:

```
.nexus/
├── memory.json           # Project identity & focus
├── architecture.json     # Layers, tech, important files
├── timeline.json         # Chronological events
├── decisions.json        # Architectural decisions (ADR)
├── tasks.json            # Work tracking
├── sessions/            # AI session records
├── snapshots/           # Memory checkpoints
├── knowledge.db          # SQLite + FTS5 search + graph
└── project.nxp          # Portable Nexus Protocol export
```

5 Command Reference

All 16 commands. Global option `-p, --path <PATH>` selects the project directory (defaults to the current directory).

Command	Description
<code>nexus init</code>	Create the <code>.nexus/</code> memory layer in the current project
<code>nexus scan</code>	Analyze source code, dependencies, and git history
<code>nexus continue</code>	Flagship — reconstruct complete project understanding
<code>nexus handoff</code>	Generate a universal AI continuation handoff prompt
<code>nexus snapshot</code>	Create a memory checkpoint
<code>nexus restore</code>	Restore a previous memory checkpoint
<code>nexus timeline</code>	Display chronological project evolution
<code>nexus decide</code>	Record an architectural decision
<code>nexus decisions</code>	Display all stored decisions
<code>nexus ask</code>	Semantic / full-text search across project memory
<code>nexus health</code>	Analyze project health score
<code>nexus map</code>	Generate an architecture map
<code>nexus impact</code>	Trace what a file connects to in the knowledge graph
<code>nexus export</code>	Export <code>project.nxp</code> (Nexus Protocol)
<code>nexus import</code>	Import <code>project.nxp</code>
<code>nexus session</code>	Record an AI session

Options & flags per command

Command	Flags	Notes
<code>init</code>	<code>-n, --name <NAME></code>	Defaults to the directory name
<code>scan</code>	—	Builds architecture + knowledge graph
<code>continue</code>	<code>-c, --compress</code> · <code>--max-tokens <N=8000></code> · <code>-o, --output <FILE></code> · <code>-t, --task <TEXT></code>	<code>--task</code> ranks memory by relevance to a focus task
<code>handoff</code>	<code>-c, --compress</code> · <code>--max-tokens <N=8000></code> · <code>-o, --output <FILE></code>	Emits an XML handoff envelope
<code>snapshot</code>	<code><LABEL></code> · <code>-d, --description <TEXT></code>	Label is positional & required
<code>restore</code>	<code><ID></code>	Snapshot ID prefix or label
<code>timeline</code>	<code>-l, --limit <N=50></code>	Most recent events first
<code>decide</code>	<code><CONTENT></code> · <code>-r, --rationale <TEXT></code> · <code>-t, --tag <TAG></code> (repeatable) · <code>-s, --</code>	<code>--supersedes</code> marks a prior decision (by ID prefix) as superseded

	<code>supersedes <ID></code>	
<code>decisions</code>	—	Lists all decisions chronologically
<code>ask</code>	<code><QUERY> · -l, --limit <N=10></code>	FTS5 + graph + decision search
<code>health</code>	—	Debt, dependency risk, docs, complexity, maintainability
<code>map</code>	—	Architecture tree (requires a prior scan)
<code>impact</code>	<code><FILE></code>	Path or fragment; shows connected nodes & relations
<code>export</code>	—	Writes <code>.nexus/project.nxp</code>
<code>import</code>	<code>-f, --file <PATH></code>	Defaults to <code>.nexus/project.nxp</code>
<code>session</code>	<code>-p, --prompt <P> · -r, --response <R> · -t, --tool <T> · -f, --file <F> (repeatable) · -n, --notes <N></code>	Records an AI interaction into memory

Typical workflow

```
# 1. Initialize and analyze
nexus init
nexus scan

# 2. Record knowledge as you work
nexus decide "Use SQLite for offline-first local storage" --rationale "no server dependency"
nexus decide "Use Postgres for the API" --supersedes 9b353bf8

# 3. Reconstruct context for an AI assistant
nexus continue --task "implement the search index"
nexus continue --compress --max-tokens 4000 --output context.txt
nexus handoff --output handoff.xml

# 4. Explore & maintain
nexus impact db.rs
nexus ask "authentication flow"
nexus map
nexus health
nexus snapshot "before-refactor" --description "stable checkpoint"
nexus restore before-refactor
```

6 Data & Persistence

Nexus uses **dual persistence**: human-readable JSON for portability and inspection, and SQLite for indexed querying.

SQLite (knowledge.db)

- **Tables:** decisions, tasks, timeline_events, knowledge_nodes, knowledge_edges, sessions, snapshots, scan_history, schema_meta.
- **Full-text search:** an FTS5 virtual table (`memory_fts`) with the porter tokenizer powers `nexus ask` and the `--task` ranking.
- **Durability:** WAL journal mode, `synchronous=NORMAL`, a 5-second busy timeout, and enforced foreign keys.

JSON stores

- memory.json, architecture.json, timeline.json, decisions.json, tasks.json plus per-file session records.
- **Atomic writes:** every JSON write goes to a temp file, is flushed and fsynced, then atomically renamed over the target — a crash mid-write cannot corrupt memory.

7 NXP — Nexus Protocol

The **Nexus Protocol (NXP)** is a portable, self-describing document that captures the full state of a project's memory — identity, architecture, decisions, tasks, timeline, knowledge graph, recent AI sessions, and a git summary — for transfer between machines, teammates, or tools.

- `nexus export` writes `.nexus/project.nxp`.
- `nexus import [--file <path>]` ingests an NXP document.
- Current protocol version: **1.0.0**. The handoff envelope is versioned and XML-escaped for safe embedding in any AI client.

8 Reliability & Recent Enhancements

Hardening and intelligence improvements delivered in the latest pass.

Area	Improvement	Status
Crash safety	Atomic JSON writes (temp → fsync → rename)	Done
Concurrency	SQLite WAL, busy-timeout, foreign keys	Done
Correctness	UTF-8-safe text truncation (was a panic on multibyte text)	Done
Correctness	Complete XML escaping in the handoff envelope	Done
Robustness	No panic when the working directory is unavailable	Done
Intelligence	<code>decide --supersedes</code> — decision lifecycle	Done
Intelligence	<code>nexus impact <file></code> — knowledge-graph traversal	Done
Intelligence	Graph centrality ranking of important files in <code>continue</code>	Done
Intelligence	<code>continue --task</code> — FTS5 relevance-ranked context	Done
Quality gate	Workspace builds clean; tests pass; clippy <code>-D warnings</code> clean	Done

Planned next

- Next

 MCP server mode (live agent queries)
- Next

`nexus sync` from git commits
- Next

 NXP v2 with checksums
- Next

 Optional local embeddings & vector search

9 Build, Test & Install

Build

```
# Debug build
cargo build --workspace

# Optimized release binary (~10 MB, LTO + stripped)
cargo build --release --workspace
```

Test & lint

```
cargo test --workspace
cargo clippy --workspace -- -D warnings
cargo fmt --all -- --check
```

Install

```
# Put the `nexus` binary on your PATH
cargo install --path crates/nexus-cli

# Verify
nexus --version
nexus --help
```

10 Roadmap

Phase	Theme	Highlights
v0.1 (current)	MVP	init → scan → decide → continue loop; FTS5 search; snapshots; health; NXP 1.0
v0.2	Intelligence	Session importers; symbol-level graph; auto-tasks from commits; <code>nexus watch</code> ; decision supersession ✓; shell completions
v0.3	Ecosystem	Plugin API; VS Code / Cursor extension; optional embeddings; vector search; memory diff; CI integration
v1.0	Distribution	Signed binaries; Homebrew / winget / scoop; crates.io; NXP v2 with checksums; migration tooling
v2.0	Platform	Optional self-hosted sync; cross-project linking; org decision registry; agent SDK; NXP conformance suite

Success metrics

Metric	Target
<code>nexus continue</code> latency	< 2s for a 10k-file project
Context token efficiency	~70% reduction with <code>--compress</code>
NXP round-trip fidelity	100% field preservation
Offline operation	Zero network calls
Cross-platform	Windows, macOS, Linux